



New Mansoura University
Faculty of Computer Science and Engineering



Advanced Machine Learning

End-to-End Machine Learning Project

Islam Reda

Associate Professor

islam.reda@nmu.edu.eg

End-to-End Machine Learning Project

1. Look at the big picture.
2. Get the data.
3. Discover and visualize the data to gain insights.
4. Prepare the data for Machine Learning algorithms.
5. Select a model and train it.
6. Fine-tune your model.
7. Launch, monitor, and maintain your system.

4- Prepare the Data for Machine Learning Algorithms

4- Prepare the Data for Machine Learning Algorithms

- **Data Cleaning**
  - Most Machine Learning algorithms cannot work with missing features. The total_bedrooms attribute has some missing values. To fix this, you have three options:
    1. Get rid of the corresponding districts (dropna())
    2. Get rid of the whole attribute (drop()).
    3. Set the values to some value (zero, the mean, the median, etc.) (fillna()).

```
housing.dropna(subset=["total_bedrooms"])        # option 1
housing.drop("total_bedrooms", axis=1)           # option 2
median = housing["total_bedrooms"].median()      # option 3
housing["total_bedrooms"].fillna(median, inplace=True)
```

  - If you choose option 3, you should **compute the median** value on the training set, and use it to fill the missing values in the training set, but also don't forget to save the median value that you have computed.

4- Prepare the Data for Machine Learning Algorithms

- **Data Cleaning**
  - Scikit-Learn provides a class to take care of missing values: **SimpleImputer**.

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy="median")
```

  - Since the median can only be computed on numerical attributes, we need to create a copy of the data without the text attribute ocean_proximity:

```
housing_num = housing.drop("ocean_proximity", axis=1)
```

  - You can fit the imputer instance to the training data using the fit() method:

```
imputer.fit(housing_num)
```

4- Prepare the Data for Machine Learning Algorithms

- **Data Cleaning**
  - The imputer has simply computed the median of each attribute and stored the result in its statistics_ instance variable.

```
>>> imputer.statistics_
array([ -118.51 , 34.26 , 29. , 2119.5 , 433. , 1164. , 408. , 3.5409])
```

  - You can use this "trained" imputer to transform the training set by replacing missing values by the learned medians:

```
X = imputer.transform(housing_num)
```

  - The result is a plain NumPy array containing the transformed features. If you want to put it back into a Pandas DataFrame, it's simple:

```
housing_tr = pd.DataFrame(X, columns=housing_num.columns)
```

4- Prepare the Data for Machine Learning Algorithms

- **Handling Text and Categorical Attributes**
  - Earlier we left out the categorical attribute ocean_proximity because it is a text attribute

```
>>> housing_cat = housing[["ocean_proximity"]]
>>> housing_cat.head(3)
       ocean_proximity
17606        <1H OCEAN
14650       NEAR OCEAN
3230            INLAND
```

  - Most Machine Learning algorithms prefer to work with numbers, so let's convert these categories from text to numbers using Scikit-Learn's **OrdinalEncoder** class:

```
>>> from sklearn.preprocessing import OrdinalEncoder
>>> ordinal_encoder = OrdinalEncoder()
```

4- Prepare the Data for Machine Learning Algorithms

- **Handling Text and Categorical Attributes**

```
>>> housing_cat_encoded = ordinal_encoder.fit_transform(housing_cat)
>>> housing_cat_encoded[:3]
array([[0.],
       [4.],
       [1.]])
```

- You can get the list of categories using the categories_ instance variable.

```
>>> ordinal_encoder.categories_
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
dtype=object)]
```

- One issue with this representation is that ML algorithms will assume that **two nearby values are more similar than two distant values**. This may be fine in some cases (e.g., for ordered categories such as “bad”, “average”, “good”).

4- Prepare the Data for Machine Learning Algorithms

- **Handling Text and Categorical Attributes**
  - To fix this issue, a common solution is one-hot encoding. Scikit-Learn provides a **OneHotEncoder** class to convert categorical values into one-hot vectors:

```
>>> from sklearn.preprocessing import OneHotEncoder
>>> cat_encoder = OneHotEncoder()
>>> housing_cat_1hot = cat_encoder.fit_transform(housing_cat)
>>> housing_cat_1hot
<16512x5 sparse matrix of type '<class 'numpy.float64'>'
with 16512 stored elements in Compressed Sparse Row format>
```

  - Notice that the output is a SciPy sparse matrix, instead of a NumPy array.
  - To convert it to a (dense) NumPy array, just call the toarray() method:

```
>>> housing_cat_1hot.toarray()
```

4- Prepare the Data for Machine Learning Algorithms

- **Feature Scaling**
  - With few exceptions, Machine Learning algorithms don't perform well when the input numerical attributes have very different scales. Note that scaling the target values is generally not required.
  - Two common ways: **min-max scaling (normalization)** and **standardization**.
  - Scikit-Learn provides a transformer called **MinMaxScaler** for this.
  - Scikit-Learn provides a transformer called **StandardScaler** for standardization.

4- Prepare the Data for Machine Learning Algorithms

- **Standardization in sklearn**
  - **StandardScaler**: sklearn scaler, scaling values to be centered around mean 0 with standard deviation 1

$$X_{scaled} = \frac{x - x_{mean}}{x_{std}}$$

```
from sklearn.preprocessing import StandardScaler
stdsc = StandardScaler()

raw_data = np.array([[-3.4], [4.5], [50], [24], [3.4], [1.6]])
scaled_data = stdsc.fit_transform(raw_data)
print(scaled_data.reshape(1,-1))
```

```
[[-0.90560498 -0.47848383  1.98151777  0.57580257 -0.53795639 -0.63527514]]
```

4- Prepare the Data for Machine Learning Algorithms

- **MinMax Scaling in sklearn**
  - **MinMaxScaler**: sklearn scaler, scaling values so that minimum value is 0 and maximum value is 1

$$X_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

```
from sklearn.preprocessing import MinMaxScaler
minmaaxsc = MinMaxScaler ()

raw_data = np.array([[-3.4], [4.5], [50], [24], [3.4], [1.6]])
scaled_data = minmaxsc.fit_transform(raw_data)
print(scaled_data.reshape(1,-1))
```

```
[[0.         0.14794007 1.         0.51310861 0.12734082 0.09363296]]
```

4- Prepare the Data for Machine Learning Algorithms

- **Transformation Pipelines**
  - Scikit-Learn provides the **Pipeline** class to help with sequences of transformations that need to be executed in order:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
num_pipeline = Pipeline([
        ('imputer', SimpleImputer(strategy="median")),
        ('std_scaler', StandardScaler()),
])
housing_num_tr = num_pipeline.fit_transform(housing_num)
```

  - The Pipeline constructor takes a list of name/estimator pairs defining a sequence of steps.
  - The pipeline's fit() method calls fit_transform() sequentially on all transformers, passing the output of each call to the next call.

4- Prepare the Data for Machine Learning Algorithms

- **Transformation Pipelines**
  - We have handled the categorical columns and the numerical columns separately. To use a single transformer to handle all columns, applying the appropriate transformations to each column, Scikit-Learn introduced the **ColumnTransformer** for this purpose.

```
from sklearn.compose import ColumnTransformer
num_attribs = list(housing_num)
cat_attribs = ["ocean_proximity"]

full_pipeline = ColumnTransformer([
        ("num", num_pipeline, num_attribs),
        ("cat", OneHotEncoder(), cat_attribs),
])
housing_prepared = full_pipeline.fit_transform(housing)
```

5- Select and Train a Model

5- Select and Train a Model

- **Training and Evaluating on the Training Set**

```
from sklearn.linear_model import LinearRegression
lin_reg = LinearRegression()
lin_reg.fit(housing_prepared, housing_labels)
```

- You now have a working Linear Regression model. Let's try it out on a few instances from the training set:

```
>>> some_data = housing.iloc[:5]
>>> some_labels = housing_labels.iloc[:5]
>>> some_data_prepared = full_pipeline.transform(some_data)
>>> print("Predictions:", lin_reg.predict(some_data_prepared))
Predictions: [ 210644.6045  317768.8069  210956.4333  59218.9888  189747.5584]
>>> print("Labels:", list(some_labels))
Labels: [286600.0, 340600.0, 196900.0, 46300.0, 254500.0]
```

5- Select and Train a Model

- **Training and Evaluating on the Training Set**
  - Let's measure this regression model's RMSE on the whole training set:

```
>>> from sklearn.metrics import mean_squared_error
>>> housing_predictions = lin_reg.predict(housing_prepared)
>>> lin_mse = mean_squared_error(housing_labels, housing_predictions)
>>> lin_rmse = np.sqrt(lin_mse)
>>> lin_rmse
68628.19819848922
```

  - This is clearly not a great score: most districts' median_housing_values range between $120,000 and $265,000, so a typical prediction error of $68,628 is not very satisfying.
  - This is an example of a model **underfitting** the training data. The **model is not powerful enough**.

5- Select and Train a Model

- **Training and Evaluating on the Training Set**
  - Let's train a **DecisionTreeRegressor**. This is a powerful model, capable of finding complex nonlinear relationships in the data:

```
from sklearn.tree import DecisionTreeRegressor
tree_reg = DecisionTreeRegressor()
tree_reg.fit(housing_prepared, housing_labels)
```

  - Let's evaluate it on the training set:

```
>>> housing_predictions = tree_reg.predict(housing_prepared)
>>> tree_mse = mean_squared_error(housing_labels, housing_predictions)
>>> tree_rmse = np.sqrt(tree_mse)
>>> tree_rmse
0.0
```

  - No error at all. It is much more likely that the model has badly **overfit** the data.

5- Select and Train a Model

- **Better Evaluation Using Cross-Validation**
  - Use Scikit-Learn's K-fold cross-validation:

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(tree_reg, housing_prepared, housing_labels,
                         scoring="neg_mean_squared_error", cv=10)
tree_rmse_scores = np.sqrt(-scores)
```

```
>>> def display_scores(scores):
... print("Scores:", scores)
... print("Mean:", scores.mean())
... print("Standard deviation:", scores.std())
...
>>> display_scores(tree_rmse_scores)
Scores: [70194.3368 66855.1636 72432.5824 70758.7389 71115.8823 75585.1417
70262.8613 70273.6325 75366.8795 71231.6572]
Mean: 71407.68766037929
Standard deviation: 2439.4345041191004
```

6- Fine-Tune Your Model

6- Fine-Tune Your Model

- **Grid Search**
  - You can fiddle with the **hyperparameters manually**, until you find a great combination of hyperparameter values.
  - Instead you should get Scikit-Learn's **GridSearchCV** to search for you.

```
from sklearn.model_selection import GridSearchCV
param_grid = [
  {'n_estimators': [3, 10, 30], 'max_features': [2, 4, 6, 8]},
  {'bootstrap': [False], 'n_estimators': [3, 10], 'max_features': [2, 3, 4]},
]
forest_reg = RandomForestRegressor()
grid_search = GridSearchCV(forest_reg, param_grid, cv=5,
                           scoring='neg_mean_squared_error',
                           return_train_score=True)
grid_search.fit(housing_prepared, housing_labels)
```

6- Fine-Tune Your Model

- **Grid Search**
  - You can get the best combination of parameters like this:

```
>>> grid_search.best_params_
{'max_features': 8, 'n_estimators': 30}
```

  - You can also get the best estimator directly:

```
>>> grid_search.best_estimator_
RandomForestRegressor(bootstrap=True, criterion='mse', max_depth=None,
           max_features=8, max_leaf_nodes=None, min_impurity_decrease=0.0,
           min_impurity_split=None, min_samples_leaf=1,
           min_samples_split=2, min_weight_fraction_leaf=0.0,
           n_estimators=30, n_jobs=None, oob_score=False, random_state=None,
           verbose=0, warm_start=False)
```

6- Fine-Tune Your Model

- **Grid Search**
  - The evaluation scores are also available:

```
>>> cvres = grid_search.cv_results_
>>> for mean_score, params in zip(cvres["mean_test_score"], cvres["params"]):
...         print(np.sqrt(-mean_score), params)
...
63669.05791727153  {'max_features': 2, 'n_estimators': 3}
55627.16171305252  {'max_features': 2, 'n_estimators': 10}
53384.57867637289  {'max_features': 2, 'n_estimators': 30}
57869.25504027614  {'max_features': 8, 'n_estimators': 3}
51711.09443660957  {'max_features': 8, 'n_estimators': 10}
49682.25345942335  {'max_features': 8, 'n_estimators': 30}
57490.612956065226  {'bootstrap': False, 'max_features': 4, 'n_estimators': 3}
51009.51445842374  {'bootstrap': False, 'max_features': 4, 'n_estimators': 10}
```

6- Fine-Tune Your Model

- **Randomized Search**
  - The grid search approach is fine when you are exploring relatively few combinations, but when the hyperparameter **search space is large**, it is often preferable to **use RandomizedSearchCV** instead.
  - This class can be used in much the same way as the GridSearchCV class, but instead of trying out all possible combinations, it **evaluates a given number of random combinations** by selecting a random value for each hyperparameter at every iteration.

6- Fine-Tune Your Model

- **Evaluate Your System on the Test Set**
  - Get the predictors and the labels from yourtest set, run your full_pipeline to transform the data, and evaluate the final model on the test set:

```
final_model = grid_search.best_estimator_

X_test = strat_test_set.drop("median_house_value", axis=1)
y_test = strat_test_set["median_house_value"].copy()

X_test_prepared = full_pipeline.transform(X_test)

final_predictions = final_model.predict(X_test_prepared)

final_mse = mean_squared_error(y_test, final_predictions)
final_rmse = np.sqrt(final_mse) # => evaluates to 47,730.2
```

7- Launch, Monitor, and Maintain Your System

7- Launch, Monitor, and Maintain Your System

- You need to **get your solution ready for production**, in particular by **plugging the production input data sources into your system**.
- You also need to **write monitoring code** to check your system's live performance at regular intervals and trigger alerts when it drops.
- Evaluate the system's input data quality. Sometimes performance will degrade slightly because of a poor quality signal (e.g., a malfunctioning sensor sending random values, or another team's output becoming stale).
- Finally, you will generally want to **train your models** on a regular basis **using fresh data**. You should automate this process as much as possible.

Thank You